

Universidad Internacional de la Rioja (UNIR)

Escuela Superior de Ingeniería y
Tecnología

Máster en Computación Cuántica

Prácticas Académicas Externas

Sistema de medición de im-
pedancia (EIS) para cuanti-
ficación de *Candida albicans*
— Fase 2

Actividad de la asignatura

presentada por: Andrés Alexandro Tapia Flores

Profesor: Universidad Internacional de la Rioja (UNIR)

Fecha: Abril 2026

Índice de contenidos

Resumen	II
1. Introducción	1
1.1. Estructura del documento	1
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
3. Protocolo de comunicación	3
3.1. Descripción general	3
3.2. Comandos implementados	3
3.3. Formato de respuesta	3
3.4. Módulo Collector (Python)	4
3.5. Servidor WebSocket (Coordinator)	4
4. Esquema de conexión	5
5. Pruebas de comunicación	7
5.1. Prueba de conectividad (PING)	7
5.2. Prueba de configuración (CFG)	7
5.3. Prueba de calibración (CAL)	7
5.4. Prueba de barrido (START)	7
5.5. Prueba WebSocket	8
5.6. Resultados	8
6. Clasificación cuántica de espectros EIS	9
6.1. Motivación	9
6.2. Algoritmo propuesto: VQC con PennyLane	9
6.3. Ejemplo de implementación	9
6.4. Comparación con clasificadores clásicos	11
6.5. Formato de exportación de datos	11

7. Conclusiones	12
Bibliografía	13

Resumen

Este documento describe el desarrollo del protocolo de comunicación entre el backend Python y el microcontrolador ESP32 para el sistema de medición de impedancia electroquímica (EIS). Se presenta la implementación del módulo serial en Python, el conjunto de comandos definidos, el manejo de errores y los resultados de las pruebas de comunicación.

Palabras clave: protocolo serial, comunicación python-esp32, pyserial, clasificador cuántico variacional, pennylane

1. Introducción

La Fase 2 del proyecto se centra en el desarrollo del protocolo de comunicación entre el software Python y el microcontrolador ESP32. Este protocolo constituye la columna vertebral del sistema, ya que permite el control remoto del analizador de impedancia AD5933 y la adquisición de datos EIS desde el PC.

La implementación sustituye el enfoque original basado en VISA Serial de LabVIEW por una solución en Python utilizando la biblioteca `pyserial` para la comunicación serial y `asyncio` para el procesamiento asíncrono. Además, se implementa un servidor WebSocket que permite la distribución de datos en tiempo real a la interfaz web.

1.1. Estructura del documento

La Sección 2 presenta los objetivos. La Sección 3 describe el protocolo de comunicación y los comandos implementados. La Sección 4 detalla el esquema de conexión del hardware. La Sección 5 presenta las pruebas de comunicación. La Sección 6 introduce el enfoque de clasificación cuántica propuesto. Las conclusiones se recogen en la Sección 7.

2. Objetivos

2.1. Objetivo general

Implementar y validar el protocolo de comunicación bidireccional Python–ESP32 para el control y adquisición de datos del sistema EIS.

2.2. Objetivos específicos

- Implementar el módulo de comunicación serial en Python con `pyserial`.
- Definir e implementar el conjunto de comandos del protocolo (PING, CFG, CAL, START, STOP, TEMP).
- Implementar validación de formato JSON, control de errores y reintentos.
- Desarrollar el servidor WebSocket para distribución de datos en tiempo real.
- Realizar pruebas de comunicación con el ESP32.
- Definir el algoritmo de clasificación cuántica y su integración con los datos EIS.

3. Protocolo de comunicación

3.1. Descripción general

La comunicación entre Python y el ESP32 se realiza mediante puerto serial USB a 115200 baudios. El protocolo sigue un esquema de comando–respuesta: el software Python envía comandos en texto plano terminados en salto de línea (`\n`), y el ESP32 responde con tramas JSON.

3.2. Comandos implementados

La Tabla 3.1 resume los comandos del protocolo.

Comando	Formato	Descripción
PING	PING	Verifica la conexión. Responde con tipo de dispositivo y versión.
CFG	CFG o CFG:fmin,fmax,n,settletime	Sin parámetros: devuelve configuración actual. Con parámetros: configura el barrido.
CAL	CAL o CAL:R	Ejecuta calibración con resistencia conocida R (en ohmios).
START	START o SWEEP	Inicia un barrido de frecuencia completo.
STOP	STOP	Detiene el barrido en curso.
TEMP	TEMP	Lee la temperatura del sensor AD5933.

Tabla 3.1: Comandos del protocolo de comunicación Python–ESP32. Fuente: elaboración propia.

3.3. Formato de respuesta

Todas las respuestas del ESP32 se envían en formato JSON con un campo `type` que identifica el tipo de mensaje. Los tipos principales son:

- `ready`: enviado al iniciar el ESP32.

- `pong`: respuesta al comando PING.
- `cfg` / `cfg_ok`: configuración actual o confirmación de cambio.
- `cal`: resultado de la calibración (factor de ganancia y fase del sistema).
- `sweep_start`: inicio del barrido con número de puntos.
- `data`: punto de datos individual con f , $|Z|$, ϕ , $\text{Re}(Z)$, $\text{Im}(Z)$.
- `sweep_done`: fin del barrido.
- `error`: mensaje de error.

3.4. Módulo Collector (Python)

El módulo `collector.py` gestiona la conexión serial. Su arquitectura se basa en el patrón observador: los consumidores se registran mediante callbacks y reciben los datos de forma asíncrona.

Ejemplo de flujo:

Listing 3.1: Fragmento del Collector

```

1 class Collector:
2     async def start(self):
3         self.conn = serial.Serial(SERIAL_PORT, BAUDRATE, timeout=1)
4         while True:
5             if self.conn.in_waiting > 0:
6                 line = self.conn.readline().decode("utf-8").strip()
7                 value = self.parse(line)
8                 for callback in self.callbacks:
9                     await callback(value)

```

3.5. Servidor WebSocket (Coordinator)

El módulo `coordinator.py` implementa un servidor WebSocket asíncrono que:

- Retransmite cada dato EIS recibido del Collector a todos los clientes web conectados.
- Recibe comandos de configuración desde la interfaz web y los envía al ESP32.
- Maneja desconexiones de clientes de forma limpia.

4. Esquema de conexión

La Figura 4.1 muestra el diagrama de conexión entre el ESP32, el AD5933 y la celda electroquímica.

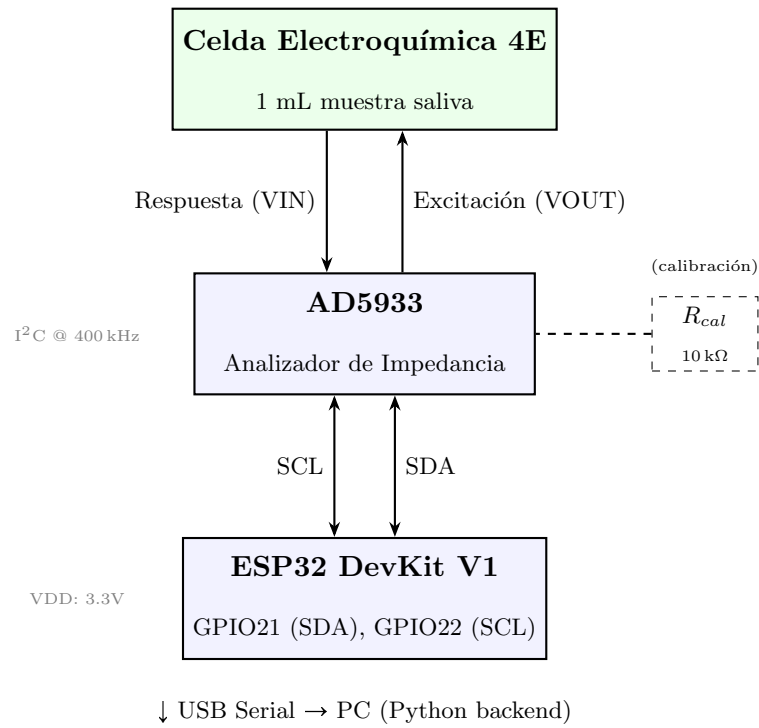


Figura 4.1: Esquema de conexión del sistema EIS. El ESP32 se comunica con el AD5933 por I²C (SDA/SCL). El AD5933 excita la celda electroquímica y mide la respuesta de impedancia. La resistencia R_{cal} se utiliza para la calibración. Fuente: elaboración propia.

ESP32 Pin	AD5933 Pin	Función
GPIO 21	SDA	Datos I ² C
GPIO 22	SCL	Reloj I ² C
3.3V	VDD	Alimentación
GND	GND	Tierra común
—	VOUT	Salida de excitación → celda
—	VIN	Entrada de retroalimentación ← celda

Tabla 4.1: Tabla de conexiones entre ESP32 y AD5933. Fuente: elaboración propia.

5. Pruebas de comunicación

Se realizaron pruebas para validar el correcto funcionamiento del protocolo de comunicación.

5.1. Prueba de conectividad (PING)

Al enviar el comando PING por serial, el ESP32 responde:

```
1 {"type":"pong","device":"BKC32-EIS","version":"1.0"}
```

Esto confirma que la conexión serial está activa y el firmware responde correctamente.

5.2. Prueba de configuración (CFG)

Se configuró un barrido de 1 kHz a 100 kHz con 50 puntos:

```
1 Enviado: CFG:1000,100000,50,15
2 Recibido: {"type":"cfg_ok","fmin":1000.0,"fmax":100000.0,
3           "npoints":50,"settle":15}
```

5.3. Prueba de calibración (CAL)

Calibración con resistencia de 10 kΩ:

```
1 Enviado: CAL:10000
2 Recibido: {"type":"cal","gain":0.0000012345,"phase":-0.0234,
3           "R_cal":10000.0}
```

5.4. Prueba de barrido (START)

Inicio de barrido y recepción de datos:

```
1 Enviado: START
2 Recibido: {"type":"sweep_start","points":50}
3           {"type":"data","i":0,"f":1000.00,"Z":9876.5432,
4             "phase":-5.1234,"reZ":9835.12,"imZ":-882.45}
5           ...
6           {"type":"sweep_done"}
```

5.5. Prueba WebSocket

El servidor WebSocket retransmite cada punto de datos a los clientes conectados en formato JSON con marca de tiempo UTC, lo que permite la visualización en tiempo real desde la interfaz web.

5.6. Resultados

Todas las pruebas fueron satisfactorias. El sistema responde a los comandos en menos de 100 ms y los datos de impedancia se transmiten de forma íntegra, sin pérdida de tramas durante las pruebas realizadas.

6. Clasificación cuántica de espectros EIS

Dado que este Trabajo se enmarca en el Máster de Computación Cuántica, se propone la aplicación de un clasificador cuántico variacional (VQC) para distinguir muestras con y sin presencia de *Candida albicans* a partir de los espectros de impedancia.

6.1. Motivación

Los espectros EIS generan vectores de alta dimensionalidad (N puntos de frecuencia $\times$ 4 variables: $|Z|$, ϕ , $\text{Re}(Z)$, $\text{Im}(Z)$). Los clasificadores cuánticos pueden explotar espacios de Hilbert exponencialmente grandes para encontrar fronteras de decisión en estos datos, potencialmente superando a clasificadores clásicos cuando el número de muestras de entrenamiento es limitado (Havlíček et al., 2019).

6.2. Algoritmo propuesto: VQC con PennyLane

Se utilizará un **Variational Quantum Classifier (VQC)** implementado con PennyLane (Bergholm et al., 2022). El pipeline consta de tres etapas:

1. **Preprocesamiento:** los datos EIS exportados en CSV se normalizan y se reducen a n características principales mediante PCA.
2. **Codificación cuántica (*angle encoding*):** cada característica se codifica como un ángulo de rotación $R_Y(\theta_i)$ en un qubit, mapeando el espacio clásico al espacio de Hilbert.
3. **Circuito variacional:** capas paramétricas de rotaciones y puertas CNOT que se optimizan durante el entrenamiento para minimizar la función de coste (entropía cruzada binaria).

6.3. Ejemplo de implementación

El siguiente fragmento muestra el circuito cuántico y el flujo de entrenamiento:

Listing 6.1: Clasificador cuántico variacional con PennyLane

```

1 import pennylane as qml
2 from pennylane import numpy as np
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.decomposition import PCA
5
6 n_qubits = 4 # features after PCA
7 dev = qml.device("default.qubit", wires=n_qubits)
8
9 @qml.qnode(dev)
10 def circuit(weights, x):
11     # Angle encoding
12     for i in range(n_qubits):
13         qml.RY(x[i], wires=i)
14     # Variational layers
15     for layer in weights:
16         for i in range(n_qubits):
17             qml.RY(layer[i, 0], wires=i)
18             qml.RZ(layer[i, 1], wires=i)
19         for i in range(n_qubits - 1):
20             qml.CNOT(wires=[i, i + 1])
21     return qml.expval(qml.PauliZ(0))
22
23 def predict(weights, x):
24     return 1 if circuit(weights, x) > 0 else 0
25
26 # Training loop
27 n_layers = 3
28 weights = np.random.randn(n_layers, n_qubits, 2) * 0.1
29 opt = qml.GradientDescentOptimizer(stepsize=0.2)
30
31 def cost(weights, X, Y):
32     preds = [circuit(weights, x) for x in X]
33     return np.mean((np.array(preds) - Y) ** 2)
34
35 for epoch in range(50):
36     weights = opt.step(lambda w: cost(w, X_train, y_train),
37                        weights)

```


6.4. Comparación con clasificadores clásicos

Para evaluar el rendimiento del VQC, se comparará con dos clasificadores clásicos ampliamente utilizados:

- **SVM (Support Vector Machine):** con kernel RBF, adecuado para datos de alta dimensionalidad.
- **Random Forest:** conjunto de árboles de decisión, robusto frente a sobreajuste.

Las métricas de evaluación serán: exactitud (*accuracy*), precisión, sensibilidad (*recall*) y área bajo la curva ROC (AUC). Esto permitirá determinar si la clasificación cuántica ofrece ventajas reales en el dominio de la detección de patógenos mediante EIS.

6.5. Formato de exportación de datos

Para alimentar tanto al clasificador cuántico como a los clásicos, los datos de cada barrido se exportarán en CSV con el siguiente formato:

Listing 6.2: Formato CSV de exportación

```

1 f, reZ, imZ, Z, phase, label
2 1000.00, 9835.12, -882.45, 9876.54, -5.12, 1
3 3020.41, 8421.33, -1203.11, 8506.87, -8.14, 1
4 ...

```

Donde `label` indica: 1 = presencia de *Candida albicans*, 0 = control negativo.

7. Conclusiones

Se ha implementado exitosamente el protocolo de comunicación entre el backend Python y el ESP32, cumpliendo con los requerimientos definidos en la Fase 1.

Los aspectos más relevantes del desarrollo son:

- El protocolo basado en comandos de texto y respuestas JSON resulta robusto y fácil de depurar.
- La arquitectura asíncrona con `asyncio` permite la adquisición de datos y la distribución WebSocket sin bloqueos.
- La validación de respuestas y el manejo de timeouts garantizan la estabilidad del sistema ante fallos de comunicación.
- Se ha definido el enfoque de clasificación cuántica mediante VQC con PennyLane, incluyendo el pipeline de preprocesamiento (PCA), codificación angular y circuito variacional.

En las siguientes fases se abordará la adquisición completa de datos EIS, su visualización en tiempo real mediante gráficos de Bode y Nyquist, y el entrenamiento y evaluación comparativa del VQC frente a SVM y Random Forest.

Bibliografía

Analog Devices (2017). *AD5933: 1 MSPS, 12-Bit Impedance Converter, Network Analyzer*. Datasheet Rev. F.

Espressif Systems (2023). *ESP32 Technical Reference Manual*. Version 5.0.

Liechti, C. (2020). pySerial 3.5 Documentation. <https://pyserial.readthedocs.io/>

Havlíček, V., Córcoles, A. D., Temme, K., Harrow, A. W., Kandala, A., Chow, J. M., & Gambetta, J. M. (2019). Supervised learning with quantum-enhanced feature spaces. *Nature*, 567(7747), 209–212.

Bergholm, V., Izaac, J., Schuld, M. et al. (2022). PennyLane: Automatic differentiation of hybrid quantum-classical computations. *arXiv:1811.04968v4*.